BSc (Hons) in **Information Technology Specialized in AI**

**Faculty of Computing**

# IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

## Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

## Part 1: Practical Implementation — Building the A* Agent

### Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

### Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

### Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

## Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

**1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?**

UCS expands nodes based solely on g(n) the actual path cost from the start to the current node. It treats every direction equally and has no knowledge of where the goal is, so it expands outward in a "cost circle" until the goal is reached.

A* expands nodes based on f(n) = g(n) + h(n) the path cost so far plus an estimated cost to reach the goal. By incorporating the heuristic h(n), A* is "goal-directed": it prioritizes nodes that appear closer to the goal, dramatically reducing the number of nodes explored while still guaranteeing optimality (provided the heuristic is admissible).

**2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?**

Admissible means the heuristic never overestimates the true cost to reach the goal: $h(n) \leq h^*(n)$.

On a 4-way grid (Up/Down/Left/Right only), the minimum number of moves required to travel from any cell to the goal is exactly the sum of the horizontal and vertical displacements: $|x_1 - x_2| + |y_1 - y_2|$. Since Manhattan Distance equals this theoretical minimum, it can never overestimate the actual cost making it admissible (and also consistent).

If the heuristic were NOT admissible (i.e., it overestimated the true cost), A* could prune the optimal path prematurely. The algorithm might believe a suboptimal route looks better because the heuristic inflates the estimated cost of the true optimal route, causing A* to return a longer (non-optimal) path or miss the best solution entirely.

**3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?**

No, Manhattan Distance would no longer be admissible for 8-way movement.

With diagonal moves allowed, an agent can reduce both x and y distance simultaneously in a single step. The true optimal cost becomes less than or equal to the Manhattan value. For example, moving from (0,0) to (3,3) takes 6 steps with Manhattan (3 Right + 3 Up), but only 3 diagonal steps in 8-way movement. Manhattan Distance (6) would overestimate the true cost (3), violating admissibility.

Which metric to switch to:
If diagonal and cardinal moves have equal cost (1), use Chebyshev Distance:
$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$

If diagonal moves cost $\sqrt{2}$ and cardinal moves cost 1, use the Octile Distance:
$h(n) = \max(dx, dy) + (\sqrt{2} - 1) \cdot \min(dx, dy)$ where $dx = |x_1 - x_2|$ and $dy = |y_1 - y_2|$

**4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.**

Using only the distance to the single closest food is weak because it ignores the spatial distribution of all remaining pellets. A much stronger heuristic would be:
Minimum Spanning Tree (MST) Heuristic over all remaining food positions.

How it works:
Treat the agent's current position and all remaining food positions as nodes in a graph.

Compute the MST connecting all these nodes using Manhattan (or appropriate) edge costs.
Use the total weight of the MST as the heuristic h(n).

Why it's stronger:
The MST captures the minimum total travel required to visit all remaining food items, not just one.
It is admissible because the MST weight is a lower bound on the true cost to visit all nodes (you cannot do better than the MST).
It provides much more informed guidance than "distance to nearest food," especially when food is scattered across the grid.

Alternative simpler heuristics (weaker than MST but stronger than nearest-food):
max(distance to any food)  you must travel at least this far to reach the farthest pellet.
nearest-food + (number_of_food × some_estimate)  accounts for remaining quantity.

**Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project**

Finalize and Lock Lab 04 Documentation

**Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.**

**FACULTY OF COMPUTING**